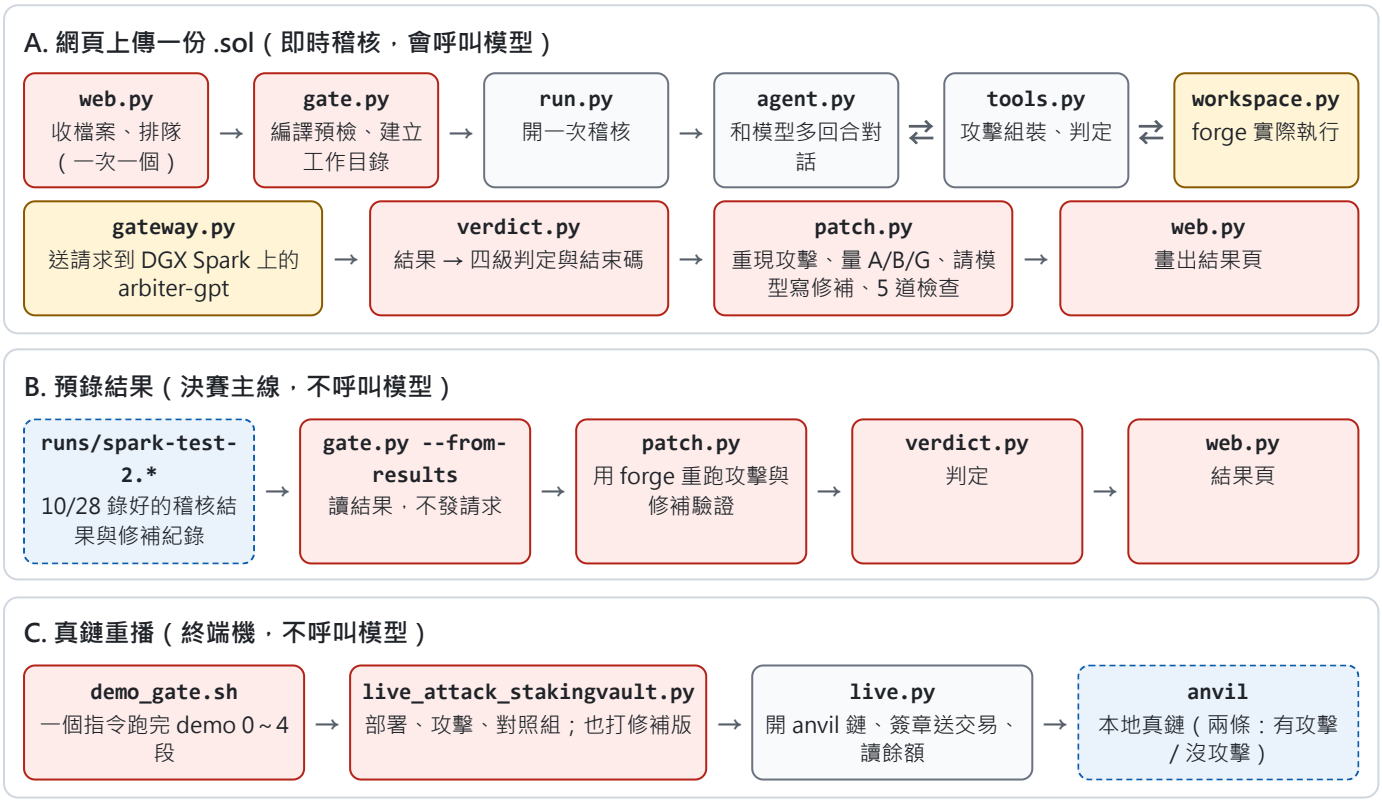
ARBITER Gate 程式地圖

每個程式在做什麼、屬於哪個時期、決賽 demo 會跑到哪些。依 WSL 本機 repo `~/innoserve/arbiter-audit`，分支 `innoserve-gate`，commit `d09b90f`（2026-10-28）整理。行數是當天的檔案行數。

> **標籤：** AIS3　AIS3 時期寫的，InnoServe 一行都沒改　AIS3 + 小改　AIS3 時期寫的，InnoServe 只做小幅、純加法的修改　新　InnoServe 新增的檔案
> 比對基準是 tag `innoserve-base`（= `ead58ae`，AIS3 結束、InnoServe 開工的那一點）。

1. 一張圖：決賽 demo 會跑到哪些檔案



灰框 = AIS3 原有、未改；黃框 = AIS3 原有、小改；紅框 = InnoServe 新增；藍虛線 = 資料或外部工具。

> **一句話分工：** AIS3 時期做出「AI 寫攻擊 → 系統判定」的核心（灰框）；InnoServe 在核心外面包了一層「部署前閘門」：判定用語、損失表、修補與驗證、網頁、demo（紅框）。核心的 `agent.py`、`tools.py` 和 `innoserve-base` 相比零差異。

2. 核心程式：`arbiter/arbiter/`

2.1 InnoServe 新增（ARBITER Gate 這一層）

檔案	行數	來源	做什麼
gate.py	464	新	閘門的命令列入口：輸入一份合約（`--file`）或一份錄好的結果（`--from-results`），輸出 `gate.json`。負責編譯預檢、repo / hermetic 模式（預設 repo）、把證實的攻擊交給 patch.py 量 A/B/G、需要時呼叫修補（`--repair`）。

`patch.py`	805	新	修補閘門。`reproduce()` 重跑原攻擊並量出 4 個環境的 A/B/G；`repair()` 請模型提出修補（最多 3 輪）；`verify_fix()` 依序做 0→L→3→1→2 五道檢查；`exploit_from_row()` 把 repo 模式的攻擊轉成單檔可跑的版本。
`verdict.py`	65	新	把一筆稽核結果對應成四級判定（已證實可攻擊 / 靜態提示未證實 / 未發現可證明攻擊 / 無法判定）和結束碼（1 / 0 / 0 / 2）。全專案只有這裡定義判定用語。
`web.py`	462	新	本機網頁（只綁 127.0.0.1:8765，只用 Python 標準函式庫）。首頁列預錄結果、提供上傳；結果頁顯示判定、A/B/G 表、修補 diff、修補後 A/B/G。網頁本身不判定，只呼叫 gate.py 再把 gate.json 畫出來。

2.2 AIS3 核心：AI 寫攻擊、系統判定

檔案	行數	來源	做什麼
`agent.py`	468	AIS3	稽核迴圈：像寫程式的 AI agent 一樣，和模型進行多回合對話，手上有編譯器和 EVM 當工具，直到證明攻擊或放棄。寫攻擊用的 prompt（`SYSTEM_PROMPT`、`USER_TEMPLATE`、`REPO_BRIEF`）都寫在這裡。
`tools.py`	1341	AIS3	模型可用的工具與執行器：`run_exploit`（組裝攻擊、放入模型看不到的成功條件、交給 forge）、`run_poc`、`submit_finding`（沒有成功的 run_exploit 就拒絕）、`conclude_safe`。判定條件（victim_loss 等）在這裡。
`workspace.py`	2234	AIS3 + 小改	每個樣本一個拋棄式 Foundry 專案，真的編譯、真的執行。包含各種防作弊閘門（不准憑空生錢、攻擊者身分晚點抽等）。InnoServe 只在 `run_poc()` 加了兩個選用參數（+17 / −2），給修補閘門用。
`gateway.py`	369	AIS3 + 小改	送請求給模型的客戶端（OpenAI 相容格式）。AIS3 時期連課程的 gateway；InnoServe 改成可用環境變數設定網址與金鑰，才能連到 DGX Spark。
`run.py`	395	AIS3	對一個評測集跑稽核：可同時跑多個、中斷可續跑，每筆都寫一行完整紀錄（判定、攻擊程式、編譯與 EVM 輸出、每一回合）。
`repo.py`	589	AIS3	合約屬於一個真實專案時，重建它的編譯環境（解析 import、相依套件），也就是 repo 模式。
`triage.py`	120	AIS3	一個專案裡哪些檔案值得稽核（跳過介面、mock、部署腳本）。gate.py 的單檔上傳**不用**它。
`reconstruct.py`	333	AIS3	把原始評測集裡無法編譯的程式片段補回最少的編譯環境，且限制補法，避免變成我們自己寫漏洞。

2.3 AIS3 證據階梯：一個發現經得起多少懷疑

檔案	行數	來源	做什麼
`prove.py`	158	AIS3	定義證據階梯：1 合成層（系統接受）→ 2 獨立專案（匯出後別處也能 forge test）→ 3 本地鏈（真的簽章交易）。
`dump.py`	326	AIS3	把證實的攻擊匯出成獨立、可直接 `forge test` 的 Foundry 專案（階梯第 2 階）。
`live.py`	262	AIS3	「盒子裡的真鏈」：啟動 anvil、發帳戶與私鑰、用簽章交易執行攻擊、從節點讀回餘額。決賽真鏈 demo 只拿它來用，沒改。
`live_replay.py`	962	AIS3	把模型證實的攻擊自動轉譯成鏈上交易重播（階梯第 3 階）。10/28 實測：StakingVault 的模型攻擊轉譯後在鏈上不成立（攻擊者拿不到資金、只重播 1 輪），所以 demo 改用 live_attack_stakingvault.py。

`onchain.py`	393	AIS3	對「已經存在的鏈狀態」（fork 模式）證明攻擊，而不是對系統自己蓋的世界。研究用，demo 不用。
`latebinding.py`	235	AIS3	攻擊者身分在模型寫完初始設定之後才抽籤，防止模型把自己設成 owner 再「證明」能掏空。

2.4 AIS3 對照組與評測（Bastet 比較、計分）

檔案	行數	來源	做什麼
`bastet_arm.py`	290	AIS3	對照組 Bastet：把它 n8n 工作流程裡的偵測提示原封不動抽出來，用同一個模型、同一個計分重跑。**整個 repo 只有這裡和 n8n 有關。**
`score.py`	348	AIS3	唯一的計分程式，所有對照組共用（F1、MCC、配對檢定 McNemar），避免各自算分灌水。
`evaluate.py`	224	AIS3	用 Bastet 自己的輸出格式計分，方便兩邊並排比較。
`benchmark.py`	262	AIS3	評測集的入場檢查：參考攻擊在有漏洞版成功、在修好版失敗、兩版都能編譯，全由 forge 驗證，沒有模型參與。
`cascade_lib.py`	180	AIS3	「先廣泛產生、再嚴格裁決」：用不同條件多跑幾次，把找到的攻擊合起來，最後一律用現行閘門重新判定。
`broadside.py`	416	AIS3	平行批次產生候選攻擊（請求數有上限，用多次獨立嘗試換召回率）。研究用。
`proposals.py`	108	AIS3	把 Bastet 個別偵測器的命中，轉成給 ARBITER 的假設清單（它的總判定沒用，但個別命中有資訊）。
`citation.py`	412	AIS3	第二級證據：EVM 無法單獨示範的缺陷（例如滑價保護設成 0），以引用程式碼的方式回報，和「可執行證據」分開標示。

3. 腳本：`arbiter/scripts/`

3.1 InnoServe 新增

檔案	行數	來源	做什麼
`demo_gate.sh`	288	新	**決賽 demo 一鍵腳本**。第 0 段前置檢查（.env、api_smoke、forge / anvil、網頁、先產生預錄頁）→ 第 1 段即時上傳（`--live`）→ 第 2 段預錄 → 第 3 段真鏈 → 第 4 段收尾。每段計時，連不上或缺工具會自動跳過。
`live_attack_stakingvault.py`	274	新	真鏈 demo：在 anvil 上部署 StakingVault，用 DRAIN 參考攻擊（7/28 為了校正判定寫的已知攻擊，**不是模型寫的**）掏空它；`--patched` 套上 arbiter-gpt 產生的修補後再打一次。結果 2/0/7 → 2/2/0，每次相同。
`live_replay_one.py`	116	新	把一筆錄好的模型攻擊交給 live_replay.py 上鏈重播（10/28 用來驗證，結果不成立，見 2.3）。
`patch_probe.py`	107	新	修補閘門的自我測試：5 種修補（2 種真修補、一律 revert、刪函式、只准外部帳戶），要被接受或拒絕，**而且要被對的那道檢查擋下**。每次 commit 前跑，5/5 as expected。
`api_smoke.py`	75	AIS3 + 小改	送一個最小請求，確認模型連得上（回 `AIS3_OK` 就是 PASS）。上台前 5 分鐘內要跑一次，閒置後第一次約 13 秒。

3.2 AIS3：防作弊探針（每次 commit 前跑的 8 個）

這些腳本不呼叫模型，專門測「判定系統會不會被騙」。每個都是正反兩向：真攻擊要接受、假攻擊要拒絕。

檔案	它在問的問題
`victim_probe.py`	問「誰虧了」能不能分出真的掏空和「拿回自己捐的錢」？（DRAIN 參考攻擊就寫在這裡）
`forgery_probe.py`	攻擊能不能偽造它本來要靠本事拿到的權限？
`halt_probe.py`	攻擊能不能靠「躲開判定條件」而不是真的滿足它來過關？
`drain_probe.py`	掏空自己布置的道具（不是別人的錢）會不會被拒絕？
`sweep_probe.py`	換不同環境重跑（餘額、時間、攻擊輪數）會不會找到真攻擊、又不捏造假攻擊？
`latebinding_probe.py`	晚點抽攻擊者身分，能不能擋下作弊的設定、又放行真攻擊？
`setup_forgery_probe.py`	初始設定能不能幫攻擊「洗」出被禁止的權限？
`gain_probe.py`	「攻擊者獲利」有沒有算攻擊者真正留下的錢？

3.3 AIS3：其他探針與研究分析（決賽不用，答辯可引用）

類別	檔案	用途
判定條件研究	`ceiling_probe.py`、`clause_ablation.py`、`collide_probe.py`、`contention_probe.py`、`economy_probe.py`、`failure_cost.py`、`gate_firing.py`、`invariant_probe.py`、`toolchain_probe.py`、`onchain_probe.py`、`context_probe.py`	成功條件每一條子句有沒有作用、閘門多常觸發、缺編譯器時會不會誤判成「沒問題」、fork 模式能不能量到真掏空等

和 Bastet 比較	`arms_table.py`、`class_table.py`、`detector_baselines.py`、`cascade.py`、`cascade_potential.py`、`adjudicability.py`、`analyst_cost.py`、`slither_arm.py`	各對照組同一份樣本並排計分、各漏洞類別的表現、幫 Bastet 調公平的門檻、分析師時間（8.7 對 28.5 分鐘）、Slither 對照組
自我審查	`audit_proofs.py`、`audit_summary.py`、`audit_labels.py`、`recheck_verdicts.py`、`regate.py`、`report_arm.py`、`blame_census.py`、`_run_census.py`	重新審查所有被接受過的證明、標準答案本身對不對、今天的閘門會不會拒絕昨天接受的攻擊、照事先登記的方式報告
滑價研究	`slippage_probe.py`、`slippage_holdout.py`	citation.py 的滑價判讀和人工標記一不一致、會不會過度擬合
真鏈	`live_attack.py`、`live_sweep.py`	AIS3 的真鏈示範（WithdrawalQueue，每次 commit 前正反兩向各跑一次）、把匯出的攻擊全部上鏈重播
執行與環境	`demo.sh`、`run_all.sh`、`launch_*.sh`、`build_dep_cache.sh`、`scan_imports.py`、`smoke_repo.py`、`derig.py`、`omp_audit.py`、`arbiter_mcp.py`	AIS3 版 demo、一鍵跑全部、背景執行、預先下載相依套件、把工具包成 MCP 給別的 agent 用等
除錯小工具	`_look.py`、`ctx_debug.py`、`rounds_debug.py`、`why_safe.py`、`build_deck.js`	看編譯錯誤、看某一輪的執行、解釋為什麼判成沒找到、產生簡報

4. 資料夾與文件

位置	內容
`arbiter/cli.py`	AIS3 的總命令列：`run`、`audit`、`fix`、`score`、`bastet`、`compare` 等子指令。`fix` 是 InnoServe Step 1b 加的。
`arbiter/examples/`	`StakingVault.sol`：我們自己寫的示範合約（unstake 先轉帳後扣帳的重入漏洞），**沒有部署在任何鏈上**。
`arbiter/evalsets/`	評測集。`v3_authored.json`：自己寫的 20 種漏洞 × 2（有漏洞版 / 修好版）= 40 份；`v1_paired14.json`：去識別化版本（9/26）。
`arbiter/runs/`	稽核紀錄（不進 git）。**spark-test-1 ~ 5 是 demo 預錄素材**，git 裡沒有，只靠備份。
`~/innoserve/runs_archive/`	10/28 從首頁搬走的 16 筆臨時紀錄（只搬不刪）。
`~/innoserve/reports/`	每一步的測試報告（Step 2a、2b、交叉驗證、eth_profit 成因、repo 模式試跑、真鏈、排練、清理……）。
`~/innoserve/CLAUDE.md`	交接與進度紀錄，**第 7 節是目前進度的唯一依據**，含 13 條已知陷阱。
`~/innoserve/DEMO_RUNBOOK.md`	決賽一頁操作手冊：開場檢查清單、每段要說的話、備案、實測耗時。
`RESULTS*.md`、`PREREGISTRATION*.md`、`HARNESS_DEFECTS.md`	AIS3 時期的成績、事先登記的實驗計畫、自己找到的系統缺陷紀錄。

5. 答辯時「這段是誰做的」快速回答

被問到	回答

判定攻擊成不成立的邏輯	AIS3 時期的 `tools.py` / `workspace.py`，InnoServe 沒改判定規則（零差異可驗證）
四級判定、「不等於安全」	InnoServe 新增的 `verdict.py`
A/B/G 損失表	判定條件 victim_loss 是 AIS3 的；把它在 4 個環境量出來、顯示成表格是 InnoServe 的 `patch.py` / `web.py`
修補與修補驗證	InnoServe 新增的 `patch.py`（唯一的新技術）
網頁	InnoServe 新增的 `web.py`（10/28）
真鏈 demo	鏈的基礎 `live.py` 是 AIS3 的；StakingVault 版腳本是 InnoServe 新寫的；攻擊是 AIS3 時期的 DRAIN 參考攻擊；修補是 arbiter-gpt 產生的
寫攻擊的 prompt	AIS3 時期 `agent.py` 裡寫死的文字，InnoServe 沒改；寫修補的 prompt 在 `patch.py`，是 InnoServe 新寫的
和 Bastet / n8n 的關係	Bastet 只是對照組，只出現在 `bastet_arm.py`；ARBITER 是另外用 Python 寫的